

```
#practice task 1
graph={
    "1" : ["2" , "3"],
    "2" :["4","5"],
    "3":["6","7"],
    "4": [],
    "5": [],
    "6": [],
    "7":[]
}

visited=[]
queue=[]

def bfs(visited,graph,node):
    visited.append(node)
    queue.append(node)

    while queue:
        current=queue.pop(0)
        print(current,"")

        for neighbour in graph[current]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

print("BFS\n")
bfs(visited,graph,"1")
```

BFS

1
2
3
4
5
6
7

```
#practice task 2
graph={
    "1":["2","3"],
    "2":["4","5"],
    "3":["6","7"],
    "4": [],
    "5": [],
    "6": [],
    "7":[]
}

visited=[]
def dfs(visited,graph,node):
    if node not in visited:
        visited.append(node)
        print(node)

        for neighbour in graph[node]:
            dfs(visited,graph,neighbour)

print("DFS\n")
dfs(visited,graph,"1")
```

DFS

1
2
4
5
3
6
7

```

from os import path
#practice task 3
Graph_nodes = {
    'A': [( 'B', 6), ( 'F', 3)],
    'B': [( 'C', 3), ( 'D', 2)],
    'C': [( 'D', 1), ( 'E', 5)],
    'D': [( 'C', 1), ( 'E', 8)],
    'E': [( 'I', 5), ( 'J', 5)],
    'F': [( 'G', 1), ( 'H', 7)],
    'G': [( 'I', 3)],
    'H': [( 'I', 2)],
    'I': [( 'E', 5), ( 'J', 3)],
}

def get_neighbors(v):
    if v in Graph_nodes:
        return Graph_nodes[v]
    else:
        return None

def h(n):
    H_dist = {
        'A': 10,
        'B': 8,
        'C': 5,
        'D': 7,
        'E': 3,
        'F': 6,
        'G': 5,
        'H': 3,
        'I': 1,
        'J': 0
    }
    return H_dist[n]

def aStarAlgo(start_node, stop_node):
    open_set=set(start_node)
    closed_set=set()
    g={}
    parents={}
    g[start_node]=0
    parents[start_node]=start_node

    while len(open_set)>0:
        n=None

        for v in open_set:
            if n==None or g[v] + h(v) < g[n] + h(n):
                n=v

        if n==stop_node or Graph_nodes[n]==None:
            pass
        else:
            for(m,weight) in get_neighbors(n):
                if m not in open_set and m not in closed_set:
                    open_set.add(m)
                    parents[m]=n
                    g[m]=g[n]+weight

            else:
                if g[m]>g[n]+weight:
                    g[m]=g[n]+weight
                    parents[m]=n
                    if m in closed_set:
                        closed_set.remove(m)
                    open_set.add(m)

        if n==None:
            print('Path does not exist')
            return None

    if n==stop_node:
        path=[]
        while parents[n]!=n:
            path.append(n)
            n=parents[n]

```

```

        path.append(start_node)
        path.reverse()
        print('Path found: {}'.format(path))
        return path

    open_set.remove(n)
    closed_set.add(n)

    print('Path does not exist')
    return None

aStarAlgo('A','J')
```

```

#task 1 BFS

from collections import deque

graph = {
    "A": ["B", "C", "H"],
    "B": ["A"],
    "C": ["A", "D"],
    "D": ["C", "E", "F"],
    "E": ["D", "G", "H"],
    "F": ["D", "G"],
    "G": ["E", "F"],
    "H": ["A", "E"]
}

visited = []
queue = deque()

def bfs(start):
    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

print("BFS Traversal:")
bfs("A")
```

```

BFS Traversal:
A B C H D E F G
```

```

#task 2 DFS

graph = {
    "A": ["B", "C", "H"],
    "B": ["A"],
    "C": ["A", "D"],
    "D": ["C", "E", "F"],
    "E": ["D", "G", "H"],
    "F": ["D", "G"],
    "G": ["E", "F"],
    "H": ["A", "E"]
}

visited = set()

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbor in graph[node]:
            dfs(neighbor)

print("DFS Traversal:")
dfs("A")
```

DFS Traversal:
A B C D E G F H

```
#task 3
from collections import deque

# Find blank position (0)
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate possible moves
def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)

    directions = [(1,0), (-1,0), (0,1), (0,-1)] # down, up, right, left

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(new_state)

    return neighbors

# BFS algorithm
def bfs(start, goal):
    queue = deque([(start, [])])
    visited = []

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        if state not in visited:
            visited.append(state)

            for neighbor in get_neighbors(state):
                queue.append((neighbor, path + [state]))

    return None

# Start and Goal matrices
start = [
    [1,2,3],
    [4,0,6],
    [7,5,8]
]

goal = [
    [1,2,3],
    [4,5,6],
    [7,8,0]
]

solution = bfs(start, goal)

print("Steps to reach goal:\n")
for step in solution:
    for row in step:
        print(row)
    print()
```

Steps to reach goal:

```
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]
```

```
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
```

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

```
#task 4
graph = {
    "Arad": ["Zerind", "Sibiu", "Timisoara"],
    "Zerind": ["Arad", "Oradea"],
    "Oradea": ["Zerind", "Sibiu"],
    "Sibiu": ["Arad", "Oradea", "Fagaras", "Rimnicu Vilcea"],
    "Timisoara": ["Arad", "Lugoj"],
    "Lugoj": ["Timisoara", "Mehadia"],
    "Mehadia": ["Lugoj", "Drobeta"],
    "Drobeta": ["Mehadia", "Craiova"],
    "Craiova": ["Drobeta", "Rimnicu Vilcea", "Pitesti"],
    "Rimnicu Vilcea": ["Sibiu", "Craiova", "Pitesti"],
    "Fagaras": ["Sibiu", "Bucharest"],
    "Pitesti": ["Rimnicu Vilcea", "Craiova", "Bucharest"],
    "Bucharest": ["Fagaras", "Pitesti", "Giurgiu", "Urziceni"]
}

visited = []
path = []

def dfs(city, goal):
    visited.append(city)
    path.append(city)

    if city == goal:
        return True

    for neighbor in graph[city]:
        if neighbor not in visited:
            if dfs(neighbor, goal):
                return True

    path.pop()
    return False

start = "Arad"
goal = "Bucharest"

dfs(start, goal)

print("Path from Arad to Bucharest:")
print(" -> ".join(path))
```

```
Path from Arad to Bucharest:
Arad -> Zerind -> Oradea -> Sibiu -> Fagaras -> Bucharest
```

```
#task 5
graph = {
    'A': [('B', 4), ('C', 3)],
    'B': [('D', 5)],
    'C': [('D', 7), ('E', 10)],
    'D': [('F', 2)],
    'E': [('F', 3)],
    'F': []
}

# Heuristic values
heuristic = {
    'A': 10,
    'B': 8,
    'C': 5,
    'D': 7,
    'E': 3,
    'F': 0
}

def a_star(start, goal):
```

```

open_set = {start}
closed_set = set()

g = {start: 0}
parent = {start: start}

while open_set:
    n = None

    for v in open_set:
        if n is None or g[v] + heuristic[v] < g[n] + heuristic[n]:
            n = v

    if n == goal:
        path = []
        while parent[n] != n:
            path.append(n)
            n = parent[n]
        path.append(start)
        path.reverse()

        print("Shortest Path:", path)
        return path

    open_set.remove(n)
    closed_set.add(n)

    for (m, weight) in graph[n]:
        if m not in open_set and m not in closed_set:
            open_set.add(m)
            parent[m] = n
            g[m] = g[n] + weight

        else:
            if g[m] > g[n] + weight:
                g[m] = g[n] + weight
                parent[m] = n

            if m in closed_set:
                closed_set.remove(m)
                open_set.add(m)

    print("Path does not exist")
    return None

a_star('A', 'F')

```

```

Shortest Path: ['A', 'C', 'E', 'F']
['A', 'C', 'E', 'F']

```

#task 6

Step 1: Create a 3x3 board

```

board = [
    [' ', ' ', ' '],
    [' ', ' ', ' '],
    [' ', ' ', ' ']
]

```

Step 2: Function to print the board

```

def print_board(b):
    for row in b:
        print('|'.join(row))
    print('-'*5)

```

Step 3: Check for a winner

```

def check_winner(b):
    # Check rows and columns
    for i in range(3):
        if b[i][0] == b[i][1] == b[i][2] != ' ':
            return b[i][0]
        if b[0][i] == b[1][i] == b[2][i] != ' ':
            return b[0][i]

    # Check diagonals
    if b[0][0] == b[1][1] == b[2][2] != ' ':
        return b[0][0]

```

```

    if b[0][2] == b[1][1] == b[2][0] != ' ':
        return b[0][2]
    # No winner
    return None

# Step 4: Check if the board is full
def is_board_full(b):
    for row in b:
        if ' ' in row:
            return False
    return True

# Step 5: Minimax function
def minimax(b, depth, is_maximizing):
    winner = check_winner(b)
    if winner == 'X':
        return 1
    elif winner == 'O':
        return -1
    elif is_board_full(b):
        return 0

    if is_maximizing:
        best_score = -float('inf')
        for i in range(3):
            for j in range(3):
                if b[i][j] == ' ':
                    b[i][j] = 'X'
                    score = minimax(b, depth + 1, False)
                    b[i][j] = ' '
                    best_score = max(score, best_score)
            return best_score
    else:
        best_score = float('inf')
        for i in range(3):
            for j in range(3):
                if b[i][j] == ' ':
                    b[i][j] = 'O'
                    score = minimax(b, depth + 1, True)
                    b[i][j] = ' '
                    best_score = min(score, best_score)
            return best_score

# Step 6: Find the best move for 'X'
def best_move(b):
    move = None
    best_score = -float('inf')
    for i in range(3):
        for j in range(3):
            if b[i][j] == ' ':
                b[i][j] = 'X'
                score = minimax(b, 0, False)
                b[i][j] = ' '
                if score > best_score:
                    best_score = score
                    move = (i, j)
    return move

# Example: Let's play one move for 'X'
print("Current Board:")
print_board(board)
x_move = best_move(board)
print(f"Best move for X: {x_move}")

# Apply the move
if x_move:
    board[x_move[0]][x_move[1]] = 'X'

print("Board after X moves:")
print_board(board)

```

```

Current Board:
| |
-----
| |
-----
| |
-----

```

```
Best move for X: (0, 0)
Board after X moves:
X| |
---
| |
---
| |
---
```